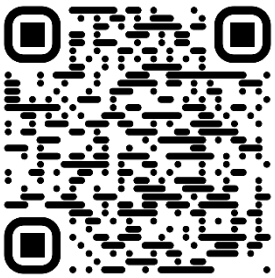
PySpark Cheatsheet 3.0



Sanjay Chandra
Microsoft Fabric CoE Community

Contents

Setup & Import

PATTERN	CODE
01 Import SparkSession and types *Always import types explicitly, avoids namespace collisions*	`from pyspark.sql import SparkSession` `from pyspark.sql.types import (` `  StructType, StructField, StringType,` `  IntegerType, DoubleType, DateType, TimestampType` `)`
02 Import functions with F alias *Industry standard, F.col(), F.when(), F.lit() and so on*	`from pyspark.sql import functions as F`
03 Import Window for ranking and running totals	`from pyspark.sql.window import Window`
04 Initialise Spark session *getOrCreate() reuses an existing session if one is running*	`spark = SparkSession.builder \` `  .appName("MyApp") \` `  .config("spark.sql.adaptive.enabled", "true") \` `  .getOrCreate()`
05 Enable adaptive query execution *AQE optimises shuffle partitions and skew joins at runtime*	`spark.conf.set("spark.sql.adaptive.enabled", "true")` `spark.conf.set("spark.sql.adaptive.coalescePartitions.enabled", "true")`
06 Check Spark version	`print(spark.version)`
07 Reduce log noise *Set to WARN in dev, ERROR in production pipelines*	`spark.sparkContext.setLogLevel("WARN")`
08 Stop the session *Always stop when done in scripts to release cluster resources*	`spark.stop()`

Key tips

F alias	always import functions as F, it is the industry standard and saves typing
getOrCreate	safe to call multiple times, returns existing session rather than creating a new one
AQE	enable adaptive query execution in all non-legacy environments for free optimisations
appName	set a meaningful app name, it appears in Spark UI and makes debugging much easier

I/O

PATTERN	CODE
01 Read Parquet *Best format for Spark, columnar, compressed and schema-aware*	`df = spark.read.parquet("path/to/data.parquet")`
02 Write Parquet	`df.write.mode("overwrite").parquet("path/to/output")`
03 Read CSV *inferSchema is slow on large files, prefer an explicit schema in production*	`df = spark.read \` `  .option("header", "true") \` `  .option("inferSchema", "true") \` `  .csv("data.csv")`
04 Read JSON *multiline required for standard nested JSON objects spanning multiple lines*	`df = spark.read.option("multiline", "true").json("data.json")`
05 Read Delta table *Delta provides versioning, ACID transactions and schema enforcement*	`df = spark.read.format("delta").load("path/to/delta_table")`
06 Write to Delta	`df.write.format("delta").mode("append").save("path/to/delta_table")` `df.write.format("delta").mode("overwrite").save("path/to/delta_table")`
07 Write partitioned data *Partition by columns used in WHERE filters for maximum query speed*	`df.write.partitionBy("year", "month").parquet("path/to/output")`
08 Save as managed table *Registers in Hive or Glue catalog, queryable with spark.sql()*	`df.write.saveAsTable("db_name.table_name")`

Key tips

`Parquet`	default to Parquet or Delta for all persistent storage, never CSV in production
`inferSchema`	always costs a full scan of the file, pass an explicit schema for large CSVs
`partitionBy`	partition on columns used in downstream filters, keeps query times fast
`overwrite`	mode overwrite replaces the entire dataset, use append to add incrementally

Schema

PATTERN	CODE
01 Define a schema explicitly *Always prefer explicit schema over inferSchema in production*	`schema = StructType([` `  StructField("id", IntegerType(), False),` `  StructField("name", StringType(), True),` `  StructField("salary", DoubleType(), True),` `  StructField("hire_date", DateType(), True),` `  StructField("updated_at", TimestampType(), True),` `])`
02 Apply schema when reading *Fastest read method, skips the inferSchema scan entirely*	`df = spark.read.schema(schema).csv("employees.csv")`
03 Print schema and column info	`df.printSchema()  # tree format` `print(df.dtypes)  # list of (name, type) tuples` `print(df.columns)  # list of column names`
04 Cast a column to a new type *Cast does not raise errors by default, invalid values become null*	`df = df.withColumn("age", F.col("age").cast(IntegerType()))` `df = df.withColumn("salary", F.col("salary").cast("double"))`
05 Force all fields to nullable *Useful when merging schemas from different sources*	`new_schema = StructType([` `  StructField(f.name, f.dataType, True)` `  for f in df.schema.fields` `])`
06 Add metadata to a field *Metadata is stored in the schema and survives write/read cycles*	`from pyspark.sql.types import MetadataBuilder` `meta = MetadataBuilder().putString("description", "monthly salary").build()` `df = df.withColumn("salary", F.col("salary").alias("salary", metadata=meta))`

Key tips

`StructField`	third argument is nullable, set False for primary keys and required fields
`cast()`	invalid values silently become null, validate after casting if data quality matters
`printSchema`	use in development to confirm types before transformations, saves debugging later
`inferSchema`	never use in production pipelines, it reads the entire file just to guess types

Nulls & Duplicates

PATTERN	CODE
01 Drop all duplicate rows *Checks every column, equivalent to SELECT DISTINCT*	`df = df.dropDuplicates()`
02 Drop duplicates on specific columns *Keeps the first occurrence, order matters if you care which row survives*	`df = df.dropDuplicates(["id"])` `df = df.dropDuplicates(["id", "dept_code"])`
03 Drop rows with any null	`df = df.na.drop(how="any")`
04 Drop rows where all columns are null	`df = df.na.drop(how="all")`
05 Drop nulls in specific columns	`df = df.na.drop(subset=["email", "phone"])`
06 Fill nulls with a default value	`df = df.na.fill(0)` `df = df.na.fill({"age": 0, "city": "Unknown", "salary": 1000.0})`
07 Replace specific values *Useful for replacing sentinel values like empty strings or -1*	`df = df.replace("", None)` `df = df.replace(-1, None, subset=["score"])`
08 Count nulls per column *Run this early as a data quality check*	`null_counts = df.select([` `  F.count(F.when(F.col(c).isNull(), c)).alias(c)` `  for c in df.columns` `])` `null_counts.show()`
09 Trim whitespace and standardise case *Do this before deduplication to catch near-duplicates*	`df = df.withColumn("name",  F.trim(F.col("name")))` `df = df.withColumn("email", F.lower(F.col("email")))`

Key tips

`dropDuplicates`	specify columns explicitly, dropping on all columns may keep unwanted duplicates
`na.fill()`	use a dict to fill different defaults per column rather than one value for all
`null counts`	always run a null count check at pipeline entry, catch data issues early
`trim + lower`	normalise strings before deduplication to catch near-duplicates like "Alice " vs "alice"

Filtering

PATTERN	CODE
01 Simple equality filter *.where() is an exact alias for .filter(), both compile identically*	`df_filt = df.filter(F.col("status") == "Active")` `df_filt = df.where(F.col("status") == "Active")  # identical`
02 Multiple conditions, AND *Parentheses are mandatory around each condition in PySpark*	`df_filt = df.filter(` `  (F.col("age") > 25) &` `  (F.col("dept") == "Sales")` `)`
03 Multiple conditions, OR	`df_filt = df.filter(` `  (F.col("category") == "A") |` `  (F.col("category") == "B")` `)`
04 Filter using a list, IN *isin() accepts *args or an unpacked list*	`df_filt = df.filter(F.col("country").isin("USA", "UK", "Canada"))`
05 Filter using a list, NOT IN *Prefix ~ to negate isin(), equivalent to SQL NOT IN*	`df_filt = df.filter(~F.col("country").isin("USA", "UK", "Canada"))`
06 Null check, is null	`df_nulls = df.filter(F.col("manager_id").isNull())`
07 Null check, is not null	`df_filt = df.filter(F.col("manager_id").isNotNull())`
08 String pattern matching *contains() / startsWith() / endsWith() / rlike() for regex*	`df_filt = df.filter(F.col("email").contains("@gmail.com"))` `df_filt = df.filter(F.col("name").startsWith("J"))` `df_filt = df.filter(F.col("name").rlike("^J.*"))  # regex`
09 Negation with tilde *~ inverts any boolean column expression*	`df_filt = df.filter(~(F.col("status") == "Closed"))`
10 Filter on a date column *Cast string columns with F.to_date() before comparing dates*	`df_filt = df.filter(F.col("hire_date") >= F.lit("2023-01-01"))` `df_filt = df.filter(` `  F.to_date(F.col("date_str"), "yyyy-MM-dd") >= F.lit("2023-01-01")` `)`
11 Filter between a range *Inclusive on both ends, equivalent to col >= low & col <= high*	`df_filt = df.filter(F.col("salary").between(50000, 100000))`

Key tips

`Parentheses`	always wrap each condition in ( ) when combining with & or |
`F.col()`	use for all column references, avoids ambiguity across joined DataFrames
`.where()`	exact alias for .filter(), both compile to the same physical execution plan
`~isin()`	cleanest NOT IN pattern, prefix isin() with a tilde to negate
`dates`	always cast with F.to_date() or F.lit() before comparing date columns

Ordering

PATTERN	CODE
01 Sort ascending (default) *orderBy() and sort() are exact aliases*	`df_sorted = df.orderBy("salary")` `df_sorted = df.sort("salary")  # identical`
02 Sort descending	`df_sorted = df.orderBy(F.col("salary").desc())`
03 Sort by multiple columns *Mix asc and desc across columns freely*	`df_sorted = df.orderBy(` `  F.col("department").asc(),` `  F.col("salary").desc()` `)`
04 Sort with nulls first *Useful for surfacing missing data at the top*	`df_sorted = df.orderBy(F.col("age").desc_nulls_first())`
05 Sort with nulls last *Default for ascending sorts*	`df_sorted = df.orderBy(F.col("age").asc_nulls_last())`
06 Sort within partitions only *No global shuffle, much faster. Use before writing files*	`df_sorted = df.sortWithinPartitions("date")`
07 Limit top N rows *Always sort before limiting to get deterministic results*	`top_5 = df.orderBy(F.col("salary").desc()).limit(5)`
08 Get first or last N rows *first() returns a Row object, not a DataFrame*	`first_row = df.first()  # single Row object` `rows  = df.take(5)  # list of Row objects` `df_head  = df.limit(5)  # DataFrame`

Key tips

`orderBy vs sort`	both are identical, orderBy is more explicit and preferred in production code
`sortWithinPartitions`	use before writing to avoid a costly global shuffle when global order is not required
`nulls placement`	specify nulls_first or nulls_last explicitly, default behaviour varies by sort direction
`limit()`	always pair with orderBy, without it limit returns arbitrary rows

Column Operations

PATTERN	CODE
01 Add a constant column *F.lit() wraps a scalar value into a column expression*	`df = df.withColumn("report_date", F.lit("2024-01-01"))` `df = df.withColumn("is_active", F.lit(True))`
02 Add a calculated column	`df = df.withColumn("total_comp", F.col("salary") + F.col("bonus"))`
03 Rename a column	`df = df.withColumnRenamed("fname", "first_name")`
04 Rename multiple columns *More efficient than chaining withColumnRenamed()*	`rename_map = {"fname": "first_name", "lname": "last_name"}` `for old, new in rename_map.items():` `  df = df.withColumnRenamed(old, new)`
05 Conditional logic, CASE WHEN *Chain as many .when() conditions as needed*	`df = df.withColumn("grade",` `  F.when(F.col("score") >= 90, "A")` `  .when(F.col("score") >= 80, "B")` `  .otherwise("C")` `)`
06 Cast data type *Cast returns null for values that cannot be converted*	`df = df.withColumn("age", F.col("age").cast(IntegerType()))` `df = df.withColumn("salary", F.col("salary").cast("double"))`
07 Concatenate columns *concat_ws ignores nulls automatically*	`df = df.withColumn("full_name",` `  F.concat_ws(" ", F.col("first"), F.col("last"))` `)`
08 Drop one or more columns	`df = df.drop("temp_column")` `df = df.drop("col_a", "col_b", "col_c")`
09 Select and reorder columns *select() can also rename via .alias()*	`df = df.select("id", "full_name", "grade")` `df = df.select(F.col("id"), F.col("salary").alias("annual_salary"))`

Key tips

`withColumn()`	creates a new DataFrame, chain sparingly on wide DataFrames as each call adds overhead
`F.lit()`	required to use a Python scalar inside a column expression, e.g. F.lit(True), F.lit(0)
`concat_ws()`	preferred over concat() because it skips null values instead of making the whole result null
`.otherwise()`	always add .otherwise() to F.when() chains, omitting it returns null for unmatched rows

String Operations

PATTERN	CODE
01 Change case	`df = df.withColumn("lower_name", F.lower(F.col("name")))` `df = df.withColumn("upper_name", F.upper(F.col("name")))` `df = df.withColumn("title_name", F.initcap(F.col("name")))`
02 Trim whitespace *Always trim before joins or comparisons*	`df = df.withColumn("clean_id", F.trim(F.col("id_str")))` `df = df.withColumn("ltrimmed",  F.ltrim(F.col("text")))` `df = df.withColumn("rtrimmed",  F.rtrim(F.col("text")))`
03 Concatenate strings *concat_ws skips nulls, concat() returns null if any arg is null*	`df = df.withColumn("address",` `    F.concat_ws(", ", F.col("city"), F.col("state"))` `)`
04 Substring extraction *Index starts at 1 in PySpark, not 0*	`df = df.withColumn("zip_prefix", F.substring(F.col("zipcode"), 1, 3))` `df = df.withColumn("year_part",  F.col("date_str").substr(1, 4))`
05 Split string into array	`df = df.withColumn("tags_array", F.split(F.col("tags"), ","))`
06 Regex replace *Remove all non-digit characters*	`df = df.withColumn("phone_clean",` `    F.regexp_replace(F.col("phone"), "[^0-9]", "")` `)`
07 Regex extract *Capture group 1 is extracted, returns empty string if no match*	`df = df.withColumn("area_code",` `    F.regexp_extract(F.col("phone"), r"^(\d{3})", 1)` `)`
08 Check string length	`df = df.withColumn("desc_len", F.length(F.col("description")))`
09 Contains, starts, ends checks	`df = df.withColumn("is_gmail", F.col("email").contains("gmail"))` `df = df.withColumn("starts_j", F.col("name").startsWith("J"))` `df = df.withColumn("ends_ltd", F.col("company").endsWith("Ltd"))`
10 Left-pad to fixed width *Useful for zero-padding IDs or codes*	`df = df.withColumn("id_padded", F.lpad(F.col("id"), 10, "0"))`

Key tips

`substr index`	PySpark substring index starts at 1, not 0, a common source of off-by-one errors
`trim first`	always trim and lower before joins or deduplication to catch near-duplicates
`regexp_extract`	returns empty string on no match, not null, filter on length > 0 to detect misses
`concat_ws()`	preferred over concat(), skips null arguments rather than propagating null

Joins

PATTERN	CODE
01 Inner join *Default join type, keeps only rows that match in both DataFrames*	`df_join = df1.join(df2, "id", "inner")`
02 Left join *Keeps all rows from left, unmatched right columns become null*	`df_join = df1.join(df2, "id", "left")`
03 Full outer join *Keeps all rows from both sides*	`df_join = df1.join(df2, "id", "outer")`
04 Left semi join *Efficient IN subquery alternative, keeps only left columns*	`df_join = df1.join(df2, "id", "left_semi")`
05 Left anti join *Efficient NOT IN alternative, rows in left with no match in right*	`df_join = df1.join(df2, "id", "left_anti")`
06 Join on multiple columns	`df_join = df1.join(df2, ["id", "dept_code"], "inner")`
07 Join on columns with different names *This keeps both key columns, drop one afterwards*	`df_join = df1.join(df2, df1["id"] == df2["emp_id"], "left")` `df_join = df_join.drop(df2["emp_id"])`
08 Broadcast join for small tables *Sends the small table to all nodes, eliminates shuffle*	`df_join = df_large.join(F.broadcast(df_small), "id")`
09 Avoid column ambiguity after join *Rename columns before joining if both DataFrames share names*	`df2 = df2.withColumnRenamed("status", "src_status")` `df_join = df1.join(df2, "id", "left")`

Key tips

`left_semi`	use instead of filter-after-inner-join when you only need columns from the left side
`left_anti`	cleanest NOT IN pattern for DataFrames, no subquery, no isin() with large lists
`broadcast()`	use for tables under ~10 MB, eliminates the shuffle that makes joins expensive
`ambiguity`	rename conflicting columns before joining, accessing df["col"] after join is fragile

Aggregations

PATTERN	CODE
01 Simple count per group	`df_agg = df.groupBy("department").count()`
02 Multiple aggregations at once *Always use .agg() for multiple calculations, more efficient*	`df_agg = df.groupBy("department").agg(` `  F.sum("salary").alias("total_salary"),` `  F.avg("salary").alias("avg_salary"),` `  F.max("bonus").alias("max_bonus"),` `  F.min("bonus").alias("min_bonus"),` `  F.count("id").alias("emp_count")` `)`
03 Count distinct values per group	`df_agg = df.groupBy("category").agg(` `  F.countDistinct("product_id").alias("unique_products")` `)`
04 Pivot, rows to columns *Specify values explicitly for better performance*	`df_pivot = df.groupBy("product").pivot("month",` `["Jan","Feb","Mar"]).sum("sales")`
05 Global aggregation, no groupBy *collect() brings result to driver, only for scalar results*	`max_val = df.agg(F.max("salary")).collect()[0][0]` `df_stats = df.agg(` `  F.mean("salary").alias("mean"),` `  F.stddev("salary").alias("stddev")` `)`
06 Collect list per group *Warning, large lists cause OOM errors, use with care*	`df_agg = df.groupBy("dept").agg(` `  F.collect_list("name").alias("employees")` `)`
07 Collect unique values per group *collect_set returns unique values only, no duplicates*	`df_agg = df.groupBy("dept").agg(` `  F.collect_set("skill").alias("unique_skills")` `)`
08 Approximate count distinct *Much faster than countDistinct on large datasets, uses HyperLogLog*	`df_agg = df.groupBy("campaign").agg(` `  F.approx_count_distinct("user_id",` `0.05).alias("approx_users")` `)`

Key tips

`.agg()`	always use .agg() for multiple aggregations in one pass, avoids redundant shuffles
`pivot()`	specify expected values explicitly, Spark scans data to find them otherwise
`collect_list()`	aggregates into an array, can cause driver OOM on large groups, use with care
`approx_count_distinct`	use instead of countDistinct on large data, orders of magnitude faster with small error

Date & Time

PATTERN	CODE
01 Get current date and timestamp	`df = df.withColumn("today", F.current_date())` `df = df.withColumn("now", F.current_timestamp())`
02 Convert string to date *Format must match the string exactly*	`df = df.withColumn("date_col", F.to_date(F.col("date_str"), "yyyy-MM-dd"))` `df = df.withColumn("ts_col", F.to_timestamp(F.col("ts_str"), "yyyy-MM-dd HH:mm:ss"))`
03 Format date as string	`df = df.withColumn("date_str", F.date_format(F.col("timestamp"), "MM/dd/yyyy"))` `df = df.withColumn("month_yr", F.date_format(F.col("date_col"), "MMM yyyy"))`
04 Date arithmetic, add and subtract *date_add and date_sub work in whole days only*	`df = df.withColumn("next_week", F.date_add(F.col("date_col"), 7))` `df = df.withColumn("last_week", F.date_sub(F.col("date_col"), 7))` `df = df.withColumn("next_month", F.add_months(F.col("date_col"), 1))`
05 Date difference in days *Returns end_date minus start_date as an integer*	`df = df.withColumn("days_diff",` `  F.datediff(F.col("end_date"), F.col("start_date"))` `)`
06 Extract date components	`df = df.withColumn("yr", F.year(F.col("date_col")))` `df = df.withColumn("mo", F.month(F.col("date_col")))` `df = df.withColumn("day", F.dayofmonth(F.col("date_col")))` `df = df.withColumn("dow", F.dayofweek(F.col("date_col")))  # 1=Sun`
07 Truncate to start of period *Useful for monthly and weekly reporting aggregations*	`df = df.withColumn("month_start", F.trunc(F.col("date_col"), "Month"))` `df = df.withColumn("year_start", F.trunc(F.col("date_col"), "Year"))` `df = df.withColumn("week_start", F.date_trunc("week", F.col("ts_col")))`
08 Unix timestamp conversion *unix_timestamp returns seconds since epoch as a long*	`df = df.withColumn("unix_ts", F.unix_timestamp(F.col("ts_col")))` `df = df.withColumn("from_unix", F.from_unixtime(F.col("unix_ts"), "yyyy-MM-dd"))`
09 Filter on date range *Wrap date literals in F.lit() for reliable comparisons*	`df_filt = df.filter(` `  (F.col("event_date") >= F.lit("2024-01-01")) &` `  (F.col("event_date") < F.lit("2025-01-01"))` `)`

Key tips

`to_date()`	always specify the format string, omitting it falls back to inference which can silently misparse
`datediff()`	argument order is (end, start), swap them and you get a negative number
`trunc()`	use for monthly aggregation keys, groups all dates in a month to a single value
`F.lit()`	wrap date string literals in F.lit() when comparing against a date column in filter()

Window Functions

PATTERN	CODE
01 Define a window specification *partitionBy groups rows, orderBy determines sequence within each group*	`from pyspark.sql.window import Window` `w_spec = Window.partitionBy("department").orderBy(F.col("salary").desc())`
02 Row number *Sequential integer, no ties, always unique within partition*	`df = df.withColumn("row_num", F.row_number().over(w_spec))`
03 Rank with gaps *Tied rows get the same rank, next rank skips (1, 2, 2, 4)*	`df = df.withColumn("rank_val", F.rank().over(w_spec))`
04 Dense rank, no gaps *Tied rows get same rank, next rank is consecutive (1, 2, 2, 3)*	`df = df.withColumn("dense_rank", F.dense_rank().over(w_spec))`
05 Lead, next row value *Returns null at partition boundary unless a default is set*	`df = df.withColumn("next_salary", F.lead("salary", 1).over(w_spec))` `df = df.withColumn("next_sal_d0", F.lead("salary", 1, 0).over(w_spec))`
06 Lag, previous row value *Returns null at partition boundary unless a default is set*	`df = df.withColumn("prev_salary", F.lag("salary", 1).over(w_spec))` `df = df.withColumn("prev_sal_d0", F.lag("salary", 1, 0).over(w_spec))`
07 Running total, cumulative sum *rowsBetween defines the frame from start of partition to current row*	`w_running = Window.partitionBy("dept").orderBy("date") \` `  .rowsBetween(Window.unboundedPreceding, Window.currentRow)` `df = df.withColumn("running_sum", F.sum("sales").over(w_running))`
08 Group average on every row *No orderBy needed for pure aggregation windows*	`w_agg = Window.partitionBy("department")` `df = df.withColumn("dept_avg", F.avg("salary").over(w_agg))`
09 Deduplicate keeping latest row *Common pattern for keeping the most recent record per key*	`w_dedup = Window.partitionBy("id").orderBy(F.col("updated_at").desc())` `df = df.withColumn("rn", F.row_number().over(w_dedup))` `df = df.filter(F.col("rn") == 1).drop("rn")`

Key tips

`row_number()`	use for deduplication, unique per partition regardless of ties
`rank vs dense`	rank() leaves gaps after ties, dense_rank() does not, choose based on reporting need
`frame clause`	rowsBetween is safer and more predictable than rangeBetween for most use cases
`w_agg`	aggregation windows without orderBy apply to the entire partition, not just preceding rows

Complex Data Types

PATTERN	CODE
01 Explode array into rows *Creates one row per element, row count increases*	`df = df.withColumn("tag", F.explode(F.col("tags_array")))`
02 Explode, keep null arrays *explode drops nulls, explode_outer keeps them as a null row*	`df = df.withColumn("tag", F.explode_outer(F.col("tags_array")))`
03 Check array contains value	`df_filt = df.filter(F.array_contains(F.col("tags_array"), "urgent"))`
04 Get array size	`df = df.withColumn("num_tags", F.size(F.col("tags_array")))`
05 Get element at index *Index is 1-based, negative index counts from the end*	`df = df.withColumn("first_tag", F.element_at(F.col("tags_array"), 1))` `df = df.withColumn("last_tag", F.element_at(F.col("tags_array"), -1))`
06 Create a map from columns	`df = df.withColumn("props", F.create_map( F.lit("color"), F.col("color_col"), F.lit("size"), F.col("size_col") ) )`
07 Get map keys and values	`df = df.withColumn("all_keys", F.map_keys(F.col("props")))` `df = df.withColumn("all_values", F.map_values(F.col("props")))`
08 Access nested struct field *Use dot notation in F.col() for nested access*	`df = df.withColumn("city", F.col("address.city"))` `df = df.withColumn("zip", F.col("address.zipcode"))`
09 Collapse columns into a struct	`df = df.withColumn("location", F.struct(F.col("city"), F.col("state"), F.col("zip")) )`
10 Collect unique values into array *collect_set returns unique values, order is not guaranteed*	`df_agg = df.groupBy("dept").agg( F.collect_set("skill").alias("unique_skills") )`

Key tips

`explode()`	increases row count, always be aware of the fan-out when exploding large arrays
`explode_outer()`	use instead of explode() when null or empty arrays must be preserved as null rows
`element_at()`	supports negative indexing for last element, array index is 1-based not 0-based
`struct()`	useful for grouping related columns before writing, struct fields survive parquet round-trips

UDFs

PATTERN	CODE

01 Standard Python UDF

Slow, serialises each row to Python. Avoid on large data

```
from pyspark.sql.functions import udf
from pyspark.sql.types import StringType

def upper_case_fn(s):
    return s.upper() if s else None

upper_udf = udf(upper_case_fn, StringType())
df = df.withColumn("upper_name", upper_udf(F.col("name")))
```

02 Register UDF as decorator

Cleaner syntax, return type annotation sets the output type

```
@udf(returnType=StringType())
def clean_phone(s):
    import re
    return re.sub(r"[^0-9]", "", s) if s else None

df = df.withColumn("phone_clean", clean_phone(F.col("phone")))
```

03 Pandas UDF, vectorised and fast

Input and output are pd.Series, much faster than standard UDF

```
from pyspark.sql.functions import pandas_udf
import pandas as pd

@pandas_udf(DoubleType())
def add_one_udf(a: pd.Series) -> pd.Series:
    return a + 1

df = df.withColumn("score_plus_one", add_one_udf(F.col("score")))
```

04 Pandas UDF with multiple input columns

```
@pandas_udf(StringType())
def combine_names(first: pd.Series, last: pd.Series) -> pd.Series:
    return first + " " + last

df = df.withColumn("full_name", combine_names(F.col("first"),
F.col("last")))
```

05 Register UDF for SQL usage

```
spark.udf.register("sql_upper", upper_case_fn, StringType())
df_sql = spark.sql("SELECT sql_upper(name) FROM employees")
```

06 Prefer built-in functions over UDFs

Native F. functions run on JVM with no serialisation cost

```
# Prefer this
df = df.withColumn("upper_name", F.upper(F.col("name")))

# Over this
df = df.withColumn("upper_name", upper_udf(F.col("name")))
```

Key tips

`pandas_udf`	10-100x faster than standard UDF, always prefer it when a native F. function is not available
`standard UDF`	serialises each row to Python, severe performance cost on large DataFrames, use sparingly
`F. first`	check pyspark.sql.functions before writing a UDF, most string, date and math ops are built in
`udf register`	register UDFs with spark.udf.register() to use them in spark.sql() queries

SCD Type 2

PATTERN	CODE
01 Filter active records in target *is_active flag identifies the current version of each record*	`active = target.filter(F.col("is_active") == "Y")`
02 Prefix source columns to avoid ambiguity	`src = source.select(` `  *[F.col(c).alias(f"src_{c}") for c in source.columns]` `)`
03 Full outer join source with active target *Full outer join catches new records and deletions in one join*	`joined = active.join(src, F.col("id") == F.col("src_id"), "full")`
04 Identify changed records *Use strict inequality to detect value changes*	`updates = joined.filter(` `  F.col("id").isNotNull() &` `  F.col("src_id").isNotNull() &` `  (F.col("salary") != F.col("src_salary"))` `)`
05 Identify brand new records *No match in target means id is null from the left side*	`new_recs = joined.filter(F.col("id").isNull())`
06 Close old records *Set is_active to N and stamp end_date for changed records*	`closed = updates.select("id", "salary", "start_date") \` `  .withColumn("end_date",  F.current_date()) \` `  .withColumn("is_active", F.lit("N"))`
07 Prepare new active records *Combine updates and net-new records into fresh active rows*	`active_new = updates.unionByName(new_recs) \` `  .select("src_id", "src_salary") \` `  .toDF("id", "salary") \` `  .withColumn("start_date", F.current_date()) \` `  .withColumn("end_date",  F.lit(None)) \` `  .withColumn("is_active",  F.lit("Y"))`
08 Union all parts into final table *Unchanged history + closed records + new active rows*	`final_df = target.join(updates, "id", "left_anti") \` `  .unionByName(closed) \` `  .unionByName(active_new)`

Key tips

is_active flag	always filter to active records before joining, avoids matching against historical rows
prefix columns	rename source columns before joining to prevent ambiguity, use a consistent prefix like src_
full outer join	catches both updates (match found) and new records (no target row) in one join
left_anti	use left_anti to carry forward unchanged history without touching those rows

Delta & Time Travel

PATTERN	CODE
01 Read Delta table *Delta provides ACID transactions, schema enforcement and versioning*	`from delta.tables import DeltaTable` `df = spark.read.format("delta").load("path/to/table")`
02 Write to Delta	`df.write.format("delta").mode("overwrite").save("path/to/table")` `df.write.format("delta").mode("append").save("path/to/table")`
03 Read a specific version *Version numbers start at 0 and increment on every write*	`df_v5 = spark.read.format("delta") \` `  .option("versionAsOf", 5) \` `  .load("path/to/table")`
04 Read as of a timestamp *Useful for point-in-time recovery or auditing*	`df_ts = spark.read.format("delta") \` `  .option("timestampAsOf", "2024-06-01 00:00:00") \` `  .load("path/to/table")`
05 View table history *Shows every operation, version, timestamp and user*	`delta_tbl = DeltaTable.forPath(spark, "path/to/table")` `delta_tbl.history().show(truncate=False)`
06 Restore to a version or timestamp *Overwrites the current table state, use with caution*	`delta_tbl.restoreToVersion(5)` `delta_tbl.restoreToTimestamp("2024-06-01")`
07 MERGE, upsert pattern *Atomic upsert, inserts new rows and updates matching rows in one operation*	`delta_tbl.alias("tgt").merge(` `  source_df.alias("src"),` `  "tgt.id = src.id"` `).whenMatchedUpdateAll() \` `  .whenNotMatchedInsertAll() \` `  .execute()`
08 Vacuum, clean up old files *Default retention is 7 days (168 hours), do not go lower without care*	`delta_tbl.vacuum(168)` `delta_tbl.vacuum(168, dry_run=True)  # check first`
09 Optimise, compact small files *Run periodically to keep read performance fast*	`delta_tbl.optimize().executeCompaction()` `delta_tbl.optimize().executeZOrderBy("event_date", "user_id")`

Key tips

`versionAsOf`	time travel back to any previous version, version 0 is the table at creation
`merge()`	atomic upsert in one pass, more efficient than separate delete and insert operations
`vacuum()`	never vacuum below 7 days if concurrent readers are active, data loss risk
`optimize()`	run after bulk loads to compact small files and keep scan performance high
`Z-order`	co-locate related data on disk by filtering columns, speeds up selective queries significantly

Streaming

PATTERN	CODE
01 Read a stream from files *Schema must be defined explicitly for file-based sources*	`df_stream = spark.readStream \` `  .schema(schema) \` `  .option("maxFilesPerTrigger", 1) \` `  .json("path/to/input_folder")`
02 Read a stream from Kafka	`df_stream = spark.readStream \` `  .format("kafka") \` `  .option("kafka.bootstrap.servers", "host:9092") \` `  .option("subscribe", "my_topic") \` `  .load()`
03 Apply transformations *Streaming DataFrames support the same API as batch DataFrames*	`df_processed = df_stream \` `  .filter(F.col("status") == "error") \` `  .withColumn("ts", F.current_timestamp())`
04 Write stream to console *For debugging only, never use console sink in production*	`query = df_processed.writeStream \` `  .outputMode("append") \` `  .format("console") \` `  .start()` `query.awaitTermination()`
05 Write stream to Delta *Checkpointing is mandatory for fault-tolerant restarts*	`query = df_processed.writeStream \` `  .format("delta") \` `  .outputMode("append") \` `  .option("checkpointLocation", "path/to/checkpoints") \` `  .start("path/to/output_table")`
06 Trigger modes *availableNow processes all backlog then stops, behaves like a batch job*	`query = df_processed.writeStream \` `  .trigger(processingTime="10 seconds").start()` `query = df_processed.writeStream \` `  .trigger(availableNow=True).start()`
07 Watermarking for late data *Drops data older than the threshold from stateful aggregations*	`df_agg = df_stream \` `  .withWatermark("timestamp", "10 minutes") \` `  .groupBy(F.window("timestamp", "5 minutes")) \` `  .count()`
08 Monitor a running query	`print(query.status)` `print(query.lastProgress)` `query.stop()`

Key tips

`checkpointLocation`	mandatory for all production sinks, stores offsets and state for fault-tolerant restarts
`availableNow`	use instead of a full batch job when you want streaming semantics with batch scheduling
`watermark`	required for stateful operations on event time, prevents unbounded state growth
`outputMode`	append adds new rows only, complete replaces the full result, update sends changed rows

Performance & Optimisation

PATTERN	CODE
01 Cache a DataFrame *Lazy, data is stored in memory only after the first action is triggered*	`df.cache()` `df.count()  # trigger the cache` `df.unpersist()  # always unpersist when done`
02 Persist to memory and disk *Prevents OOM when the DataFrame is too large for memory alone*	`from pyspark import StorageLevel` `df.persist(StorageLevel.MEMORY_AND_DISK)` `df.unpersist()`
03 Repartition, increase parallelism *Full shuffle, use to fix skew or increase parallelism before a heavy operation*	`df_repart = df.repartition(200)` `df_repart = df.repartition(200, "dept_id")  # partition by column`
04 Coalesce, reduce output files *No shuffle, only merges existing partitions. Use before writing*	`df_coal = df.coalesce(1)  # single output file` `df_coal = df.coalesce(10)  # reduce to 10 files`
05 Broadcast join for small tables *Sends small table to all workers, eliminates the shuffle entirely*	`df_join = df_large.join(F.broadcast(df_small), "id")`
06 Tune shuffle partitions *Default is 200, set to 2-3x total cores for large shuffles*	`spark.conf.set("spark.sql.shuffle.partitions", "400")` `spark.conf.set("spark.sql.adaptive.enabled", "true")`
07 Explain the execution plan *Check for BroadcastHashJoin vs SortMergeJoin and partition counts*	`df.explain()` `df.explain("extended")` `df.explain("cost")`
08 Filter and select early *Reduce data carried through the pipeline as soon as possible*	`# Good, filter reduces data before the join` `df_filtered = df.filter(F.col("year") == 2024)` `result = df_filtered.join(df2, "id")` `# Bad, join runs on full data` `result = df.join(df2, "id").filter(F.col("year") == 2024)`

Key tips

`unpersist()`	always call unpersist() when done with a cached DataFrame, frees memory for other operations
`coalesce()`	use before writing to control output file count, no shuffle makes it cheaper than repartition
`shuffle.partitions`	reduce below 200 for small data, too many partitions causes overhead from tiny tasks
`filter early`	push filters and column selects as early as possible, reduces data carried through the pipeline
`explain()`	read the physical plan before tuning, confirms whether broadcast joins and pruning are applied

Debugging

PATTERN	CODE
01 Inspect the execution plan *Look for unexpected SortMergeJoins, missing filters and skewed partitions*	`df.explain(mode="extended")  # full logical and physical plan` `df.explain(mode="formatted")  # human-readable formatted plan`
02 Show data without truncation	`df.show(n=20, truncate=False)` `df.show(n=5,  truncate=100)`
03 Check data types	`df.printSchema()` `print(df.dtypes)`
04 Check for data skew across partitions *Large variance in partition sizes causes slow tasks and stragglers*	`partition_counts = df.rdd.glom().map(len).collect()` `print(f"min: {min(partition_counts)}, max: {max(partition_counts)}")` `print(f"partitions: {df.rdd.getNumPartitions()}")`
05 Identify skewed join keys *Keys with millions of rows will slow or crash a SortMergeJoin*	`df.groupBy("join_key") \` `  .count() \` `  .orderBy(F.col("count").desc()) \` `  .show(20)`
06 Count nulls per column *Run as a first-pass data quality check at pipeline entry*	`null_counts = df.select([` `  F.count(F.when(F.col(c).isNull(), c)).alias(c)` `  for c in df.columns` `])` `null_counts.show()`
07 Profile basic statistics *describe() works on numeric and string columns*	`df.describe().show()` `df.describe("salary", "age").show()`
08 Clear cache *Call when iterative development fills up executor memory*	`spark.catalog.clearCache()`
09 Check and set Spark configuration	`print(spark.conf.get("spark.sql.shuffle.partitions"))` `spark.conf.set("spark.sql.shuffle.partitions", "100")`

Key tips

`explain()`	always check the plan before tuning, confirms whether optimisations like broadcast are applied
`skew detection`	groupBy on the join key and count, any key with far more than average rows will cause a slow task
`describe()`	quick sanity check on numeric ranges, catches nulls, zeros and unexpected outliers fast
`clearCache()`	call when iterative development fills up memory, restores full executor memory immediately
`null counts`	run the null count check at every pipeline stage boundary, catch bad data before it propagates

I/O: Cloud Storage

PATTERN	CODE
01 Read from Amazon S3 *Use s3a:// protocol, s3:// is legacy and slower*	`# Set credentials via Hadoop config or IAM role` `spark.sparkContext._jsc.hadoopConfiguration().set(` `  "fs.s3a.access.key", "YOUR_ACCESS_KEY"` `)` `spark.sparkContext._jsc.hadoopConfiguration().set(` `  "fs.s3a.secret.key", "YOUR_SECRET_KEY"` `)` `df = spark.read.parquet("s3a://my-bucket/path/to/data")`
02 Read from Azure Data Lake (ADLS Gen2) *Use abfss:// for secure TLS connection*	`spark.conf.set(` `  "fs.azure.account.key.<account>.dfs.core.windows.net",` `  "YOUR_STORAGE_KEY"` `)` `df = spark.read.parquet(` `  "abfss://<container>@<account>.dfs.core.windows.net/path"` `)`
03 Authenticate ADLS with service principal *Preferred for production, avoids storing account keys*	`spark.conf.set("fs.azure.account.auth.type.<account>.dfs.core.windows.net", "OAuth")` `spark.conf.set("fs.azure.account.oauth.provider.type.<account>.dfs.core.windows.net",` `  "org.apache.hadoop.fs.azurebfs.oauth2.ClientCredsTokenProvider")` `spark.conf.set("fs.azure.account.oauth2.client.id.<account>.dfs.core.windows.net",  "<client-id>")` `spark.conf.set("fs.azure.account.oauth2.client.secret.<account>.dfs.core.windows.net","<secret>")` `spark.conf.set("fs.azure.account.oauth2.client.endpoint.<account>.dfs.core.windows.net",` `  "https://login.microsoftonline.com/<tenant-id>/oauth2/token")`
04 Read from Google Cloud Storage *Requires the GCS connector JAR on the classpath*	`df = spark.read.parquet("gs://my-bucket/path/to/data")`
05 Write to cloud storage with partitioning *Same API as local writes, just use the cloud path*	`df.write \` `  .partitionBy("year", "month") \` `  .mode("overwrite") \` `  .parquet("s3a://my-bucket/output/")`
06 Read multiple paths at once *Spark unions the files automatically*	`df = spark.read.parquet(` `  "s3a://bucket/year=2023/",` `  "s3a://bucket/year=2024/"` `)` `# Or use a wildcard`

PATTERN	CODE
07 Use Databricks secrets for credentials *Never hard-code credentials in notebooks or scripts*	`df = spark.read.parquet("s3a://bucket/year=202*/")` `storage_key = dbutils.secrets.get(` `  scope="my-scope", key="storage-account-key"` `)` `spark.conf.set(` `  "fs.azure.account.key.<account>.dfs.core.windows.net",` `  storage_key` `)`
08 Check and list cloud files	`# Databricks` `files = dbutils.fs.ls("s3a://my-bucket/path/")` `for f in files:` `  print(f.name, f.size)` `# Standard PySpark via Hadoop FileSystem` `hadoop_fs = spark._jvm.org.apache.hadoop.fs.FileSystem` `path = spark._jvm.org.apache.hadoop.fs.Path("s3a://my-bucket/path/")` `fs = hadoop_fs.get(spark._jsc.hadoopConfiguration())` `print(fs.exists(path))`

Key tips

s3a://	always use s3a:// not s3://, it supports large files and is significantly faster
abfss://	use abfss:// (secure) not wasbs://, the wasbs protocol is legacy and deprecated
service principal	prefer service principal auth over storage account keys in all production workloads
secrets	never hard-code credentials, use Databricks secrets, AWS Secrets Manager or Azure Key Vault
wildcards	use path wildcards to read multiple partitions in one call rather than looping and unioning

Delta: Advanced MERGE

PATTERN	CODE
01 Basic upsert, update and insert *Most common MERGE pattern, handles new and changed rows*	`from delta.tables import DeltaTable` `delta_tbl = DeltaTable.forPath(spark, "path/to/table")` `delta_tbl.alias("tgt").merge(` `  source_df.alias("src"),` `  "tgt.id = src.id"` `).whenMatchedUpdateAll() \` `  .whenNotMatchedInsertAll() \` `  .execute()`
02 Conditional update, only when values change *Avoids rewriting rows that have not actually changed*	`delta_tbl.alias("tgt").merge(` `  source_df.alias("src"),` `  "tgt.id = src.id"` `).whenMatchedUpdate(` `  condition="tgt.salary != src.salary OR tgt.dept != src.dept",` `  set={"salary": "src.salary", "dept": "src.dept", "updated_at": "current_date()"}` `).whenNotMatchedInsertAll() \` `  .execute()`
03 Delete matched rows *Use for hard deletes or expiring records*	`delta_tbl.alias("tgt").merge(` `  deletes_df.alias("src"),` `  "tgt.id = src.id"` `).whenMatchedDelete() \` `  .execute()`
04 Full SCD Type 1, update in place *Overwrites history, no version tracking*	`delta_tbl.alias("tgt").merge(` `  source_df.alias("src"),` `  "tgt.id = src.id"` `).whenMatchedUpdateAll() \` `  .whenNotMatchedInsertAll() \` `  .whenNotMatchedBySourceDelete() \` `  .execute()`
05 Insert only, skip existing rows *Idempotent load pattern, safe to re-run*	`delta_tbl.alias("tgt").merge(` `  source_df.alias("src"),` `  "tgt.id = src.id AND tgt.event_date = src.event_date"` `).whenNotMatchedInsertAll() \` `  .execute()`
06 Schema evolution on MERGE *mergeSchema allows new columns in source to be added to target*	`delta_tbl.alias("tgt").merge(` `  source_df.alias("src"),` `  "tgt.id = src.id"` `).whenMatchedUpdateAll() \` `  .whenNotMatchedInsertAll() \` `  .option("mergeSchema", "true") \` `  .execute()`

PATTERN	CODE
07 Overwrite schema entirely *Use when target schema must be replaced, not extended*	`df.write.format("delta") \` `  .mode("overwrite") \` `  .option("overwriteSchema", "true") \` `  .save("path/to/table")`
08 MERGE using SQL syntax *Equivalent to the DataFrame API, useful in notebooks*	`spark.sql("""` `  MERGE INTO target_table AS tgt` `  USING source_table AS src` `  ON tgt.id = src.id` `  WHEN MATCHED THEN UPDATE SET *` `  WHEN NOT MATCHED THEN INSERT *` `""")`

Key tips

`whenMatchedUpdate`	use a condition to skip unchanged rows, avoids rewriting and speeds up MERGE significantly
`insert only`	safest idempotent load pattern, re-running never creates duplicates
`mergeSchema`	set on the MERGE call to allow new columns in source to flow through to target automatically
`overwriteSchema`	only use when fully replacing the schema, it drops columns not in the new DataFrame
`NOT MATCHED BY SOURCE`	available in Delta 2.0+ for deleting rows in target that have no match in source

Performance: Skew Handling

PATTERN	CODE
01 Detect skewed partitions *Any partition with far more rows than average is skewed*	`partition_counts = df.rdd.glom().map(len).collect()` `print(f"min: {min(partition_counts)}")` `print(f"max: {max(partition_counts)}")` `print(f"avg: {sum(partition_counts)/len(partition_counts):.0f}")`
02 Detect skewed join keys *Keys with millions of rows dominate shuffle and cause straggler tasks*	`df.groupBy("join_key") \` `  .count() \` `  .orderBy(F.col("count").desc()) \` `  .show(10)`
03 Enable AQE skew join optimisation *AQE splits large partitions automatically at runtime, no code change needed*	`spark.conf.set("spark.sql.adaptive.enabled", "true")` `spark.conf.set("spark.sql.adaptive.skewJoin.enabled", "true")` `# Tune the threshold (default 256MB)` `spark.conf.set("spark.sql.adaptive.skewJoin.skewedPartitionThresholdInBytes", "268435456")`
04 Salt skewed join key manually *Distributes hot keys across multiple partitions by adding a random suffix*	`import random` `# Add salt to the large DataFrame` `num_buckets = 10` `df_large = df_large.withColumn("salt",` `  (F.rand() * num_buckets).cast("int")` `)` `df_large = df_large.withColumn("salted_key",` `  F.concat(F.col("join_key"), F.lit("_"), F.col("salt"))` `)` `# Explode the small DataFrame to match all salt values` `df_small = df_small.withColumn("salt_range",` `  F.array([F.lit(i) for i in range(num_buckets)])` `)` `df_small = df_small.withColumn("salt", F.explode(F.col("salt_range")))` `df_small = df_small.withColumn("salted_key",` `  F.concat(F.col("join_key"), F.lit("_"), F.col("salt"))` `)` `# Join on salted key` `result = df_large.join(df_small, "salted_key", "left")`
05 Repartition on join key before joining	`df1 = df1.repartition(200, "join_key")` `df2 = df2.repartition(200, "join_key")` `result = df1.join(df2, "join_key")`

PATTERN	CODE
Pre-partitions data so shuffle during join is minimal	
06 Broadcast the small side *Eliminates shuffle entirely when one side is small enough*	`# Increase broadcast threshold if needed (default 10MB)` `spark.conf.set("spark.sql.autoBroadcastJoinThreshold", "52428800")  # 50MB` `result = df_large.join(F.broadcast(df_small), "join_key")`
07 Isolate and handle the hot key separately *Split processing so the hot key does not slow down all other keys*	`hot_key = "US"` `# Process hot key with broadcast` `df_hot = df_large.filter(F.col("country") == hot_key)` `result_hot = df_hot.join(F.broadcast(df_small), "country")` `# Process rest normally` `df_rest = df_large.filter(F.col("country") != hot_key)` `result_rest = df_rest.join(df_small, "country")` `result = result_hot.unionByName(result_rest)`

Key tips

AQE skew join	enable first, it handles most skew automatically with no code change required
salting	use when AQE is not available or skew is severe, adds complexity so only use when needed
broadcast	the simplest skew fix when the small side fits in memory, always try this first
hot key split	isolate known hot keys and handle them separately when a single key dominates the dataset
repartition	pre-partition both sides on the join key to reduce shuffle cost during the join itself

Error Handling & Logging

PATTERN	CODE
01 Handle bad records in CSV and JSON reads *PERMISSIVE mode stores corrupt records in a special column*	`df = spark.read \` `  .option("mode", "PERMISSIVE") \` `  .option("columnNameOfCorruptRecord", "_corrupt_record") \` `  .schema(schema) \` `  .csv("data.csv")` `# Inspect bad rows` `bad_rows = df.filter(F.col("_corrupt_record").isNotNull())` `bad_rows.show(truncate=False)`
02 Drop malformed records *Silently skips rows that do not match the schema*	`df = spark.read \` `  .option("mode", "DROPMALFORMED") \` `  .schema(schema) \` `  .csv("data.csv")`
03 Route bad records to a separate path *Saves corrupt records to a folder for inspection without failing the job*	`df = spark.read \` `  .option("badRecordsPath", "s3a://bucket/bad_records/") \` `  .schema(schema) \` `  .json("data.json")`
04 Try/except around a Spark action *Wrap actions, not transformations, as transformations are lazy*	`try:` `  df.write.mode("overwrite").parquet("path/to/output")` `  print("Write succeeded")` `except Exception as e:` `  print(f"Write failed: {e}")` `  raise`
05 Set up Python logging in a pipeline *Use logging instead of print() for structured, level-filtered output*	`import logging` `logging.basicConfig(` `  level=logging.INFO,` `  format="%(asctime)s %(levelname)s %(message)s"` `)` `logger = logging.getLogger(__name__)` `logger.info(f"Row count before filter: {df.count()}")` `logger.warning("Null values detected in key column")` `logger.error("Schema mismatch, aborting pipeline")`
06 Assert row count at pipeline checkpoints *Fail fast if data volumes look wrong rather than propagating bad data*	`def assert_count(df, min_rows, label=""):` `  count = df.count()` `  if count < min_rows:` `  raise ValueError(` `  f"{label}: expected >= {min_rows} rows, got {count}"` `  )` `  logger.info(f"{label}: {count} rows passed")` `  return df`

PATTERN	CODE
07 Validate schema matches expected *Catch schema drift early before it breaks downstream transforms*	`df = assert_count(df, 1000, "after filter")` `def assert_schema(df, expected_schema, label=""):` `  actual = set((f.name, str(f.dataType)) for f in df.schema.fields)` `  expected = set((f.name, str(f.dataType)) for f in expected_schema.fields)` `  missing = expected - actual` `  if missing:` `  raise ValueError(f"{label}: missing fields {missing}")` `  logger.info(f"{label}: schema validated")` `assert_schema(df, schema, "raw ingestion")`
08 Graceful pipeline with finally block *Ensures the Spark session is always stopped even on failure*	`try:` `  df = spark.read.parquet("path/to/input")` `  df = df.filter(F.col("status") == "active")` `  df.write.mode("overwrite").parquet("path/to/output")` `except Exception as e:` `  logger.error(f"Pipeline failed: {e}")` `  raise` `finally:` `  spark.stop()` `  logger.info("Spark session stopped")`

Key tips

PERMISSIVE	captures bad rows in _corrupt_record column rather than failing, always inspect this column
badRecordsPath	routes corrupt records to a separate location, keeps the main pipeline running without data loss
try/except	wrap actions not transformations, transformations are lazy and never throw at definition time
logging	use Python logging module instead of print() for level control and structured log output
assert_count	validate row counts at each stage boundary, catch data loss before it reaches production